PRACTICAL - 11

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Save/load: make a small class Serializable, save an array of its objects to a file with Object Output Stream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.
CODE	

```
import java.io.*;

class Student implements Serializable {

   private static final long serialVersionUID = 1L;

   int rollNo;
   String name;

   // This field will NOT be saved
   transient String password;

   Student(int rollNo, String name, String password) {
      this.rollNo = rollNo;
      this.name = name;
      this.password = password;
   }

   void display() {
      System.out.println(
         "Roll No: " + rollNo +
         ", Name: " + name +
         ", Password: " + password
      );
   }
}

public class SaveLoad {

   public static void main(String[] args) {

      Student[] students = {
         new Student(1, "Om", "om123"),
         new Student(2, "Rahul", "rahul123"),
         new Student(3, "Amit", "amit123")
      };

      // Save objects
      try (ObjectOutputStream out =
```

```
            new ObjectOutputStream(
              new FileOutputStream("students.dat"))) {

        out.writeObject(students);

        System.out.println("Students saved successfully.");

      } catch (IOException e) {
        e.printStackTrace();
      }

      // Load objects
      try (ObjectInputStream in =
            new ObjectInputStream(
              new FileInputStream("students.dat"))) {

        Student[] loadedStudents = (Student[]) in.readObject();

        System.out.println("\nStudents loaded from file:");

        for (Student s : loadedStudents) {
          s.display();
        }

      } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
      }
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP> cd "Practical 11"
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 11> javac SaveLoad.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 11> java SaveLoad
Students saved successfully.

Students loaded from file:
Roll No: 1, Name: Om, Password: null
Roll No: 2, Name: Rahul, Password: null
Roll No: 3, Name: Amit, Password: null
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 11>
```

PROBLEM2

2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

CODE

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;

public class LogAnalyzer {

   public static void main(String[] args) {

      String keyword = "ERROR";

      Path[] files = {
         Paths.get("log1.txt"),
         Paths.get("log2.txt"),
         Paths.get("log3.txt")
      };

      int totalLines = 0;
      int keywordLines = 0;

      try {

         for (Path file : files) {

            // Get file size
            long size = Files.size(file);

            System.out.println("\nFile: " + file.getFileName());
            System.out.println("Size: " + size + " bytes");

            // Read file line by line
            List<String> lines = Files.readAllLines(file);

            for (String line : lines) {

               totalLines++;

               if (line.contains(keyword)) {
                  keywordLines++;
               }
            }
         }

         System.out.println("\n----- SUMMARY -----");
         System.out.println("Total lines: " + totalLines);
         System.out.println(
            "Lines containing '" + keyword + "': "
            + keywordLines
```

```
        );

      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 11> javac LogAnalyzer.java
>> java LogAnalyzer

File: log1.txt
Size: 86 bytes

File: log2.txt
Size: 59 bytes

File: log3.txt
Size: 55 bytes

----- SUMMARY -----
Total lines: 10
Lines containing 'ERROR': 3
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 11>
```

PROBLEM3

3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

CODE

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.stream.Stream;

public class FolderReport {

   public static void main(String[] args) {

      Path folder = Paths.get("myfolder");
      Path report = Paths.get("folder_report.txt");

      try {

         try (Stream<Path> paths = Files.walk(folder)) {

            StringBuilder result = new StringBuilder();

            result.append("FOLDER REPORT\n");
            result.append("==================\n\n");
```

```
paths.filter(Files::isRegularFile)
    .forEach(path -> {

   try {

      BasicFileAttributes attributes =
         Files.readAttributes(
            path,
            BasicFileAttributes.class
         );

      result.append(
         "File Name: "
         + path.getFileName()
         + "\n"
      );

      result.append(
         "Path: "
         + path
         + "\n"
      );

      result.append(
         "Size: "
         + attributes.size()
         + " bytes\n"
      );

      result.append(
         "Modified: "
         + attributes.lastModifiedTime()
         + "\n"
      );

      result.append(
         "--------------------\n"
      );

   } catch (IOException e) {
      System.out.println(
         "Could not read: " + path
      );
   }
});

Files.writeString(report, result.toString());
```

```
            }

            System.out.println(
               "Folder report created successfully."
            );

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT

